

Oracle PL/SQL Collections

Associative Arrays, Nested Tables & VARRAYs · Oracle 19c

A **collection** is an ordered group of elements of the same type, accessed by an index. Oracle has three kinds: **Associative Arrays** (index-by tables), **Nested Tables**, and **VARRAYs**. They are the foundation for bulk processing (**BULK COLLECT** / **FORALL**).

1. The Three Collection Types

Feature	Associative Array	Nested Table	VARRAY
Index type	Integer or string	Integer (dense on create)	Integer
Max size	Unbounded	Unbounded	Fixed maximum
Stored in DB?	No (PL/SQL only)	Yes (column type)	Yes (column type)
Dense/Sparse	Either	Can be sparse (after delete)	Always dense
Best for	Lookups, in-memory maps	Large sets, DB storage	Small bounded lists

2. Associative Array (Index-By Table)

```
DECLARE
  -- indexed by integer
  TYPE t_nums IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  l_nums t_nums;

  -- indexed by string (like a map/dictionary)
  TYPE t_map IS TABLE OF VARCHAR2(50) INDEX BY VARCHAR2(20);
  l_map t_map;
BEGIN
  l_nums(1) := 100; l_nums(2) := 200;
  l_map('AE') := 'United Arab Emirates';
  l_map('IN') := 'India';

  DBMS_OUTPUT.put_line(l_nums(2));      -- 200
  DBMS_OUTPUT.put_line(l_map('IN'));    -- India
END;
/
```

3. Nested Table

```
DECLARE
  TYPE t_names IS TABLE OF VARCHAR2(50);
  l_names t_names := t_names('Ali', 'Sara', 'John');  -- must initialize
BEGIN
  l_names.EXTEND;      -- make room for one more
  l_names(4) := 'Maya';

  FOR i IN l_names.FIRST .. l_names.LAST LOOP
    DBMS_OUTPUT.put_line(i || ': ' || l_names(i));
  END LOOP;
END;
/
```

As a database column type:

```
CREATE OR REPLACE TYPE phone_list AS TABLE OF VARCHAR2(20);
/
CREATE TABLE contacts (id NUMBER, phones phone_list)
  NESTED TABLE phones STORE AS contacts_phones;
```

4. VARRAY (Variable-Size Array)

```
DECLARE
  TYPE t_top3 IS VARRAY(3) OF VARCHAR2(30);
  l_top t_top3 := t_top3('Gold', 'Silver');
BEGIN
  l_top.EXTEND;
  l_top(3) := 'Bronze';           -- 4th would raise: exceeds VARRAY limit
  DBMS_OUTPUT.put_line(l_top.COUNT); -- 3
END;
/
```

5. Collection Methods

Method	Purpose	Example
COUNT	Number of elements	l.COUNT
FIRST / LAST	Lowest / highest index	l.FIRST, l.LAST
NEXT(n) / PRIOR(n)	Next/previous index (sparse-safe)	l.NEXT(i)
EXISTS(n)	TRUE if element n exists	IF l.EXISTS(3)
EXTEND[(n)]	Add element slots (nested/varray)	l.EXTEND(5)
TRIM[(n)]	Remove elements from end	l.TRIM(2)
DELETE[(n[,m])]	Remove element(s)	l.DELETE(3)
LIMIT	Max size (VARRAY only)	l.LIMIT

Safe iteration over a sparse collection

```
DECLARE
  TYPE t IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  l t; i PLS_INTEGER;
BEGIN
  l(1):=10; l(5):=50; l(9):=90;           -- sparse (gaps)
  i := l.FIRST;
  WHILE i IS NOT NULL LOOP
    DBMS_OUTPUT.put_line(i || '=' || l(i));
    i := l.NEXT(i);                       -- skips gaps safely
  END LOOP;
END;
/
```

6. Collection of Records (common pattern)

```
DECLARE
  TYPE t_emp IS TABLE OF employees%ROWTYPE INDEX BY PLS_INTEGER;
  l_emp t_emp;
BEGIN
  SELECT * BULK COLLECT INTO l_emp FROM employees WHERE ROWNUM <= 5;
  FOR i IN 1 .. l_emp.COUNT LOOP
    DBMS_OUTPUT.put_line(l_emp(i).employee_id || ' ' || l_emp(i).last_name);
  END LOOP;
END;
/
```

Key points

- **Associative arrays** need no initialization; **nested tables/VARRAYs** must be initialized with a constructor before use, then **EXTENDED**.

- Use `FIRST..LAST` with a `WHILE + NEXT` for sparse collections to avoid `NO_DATA_FOUND`.
- Only **nested tables** and **VARRAYs** can be stored in database columns; associative arrays are PL/SQL-only.
- Collections are the basis for **BULK COLLECT** and **FORALL** — see the Bulk Collection guide.